

Metr User Service — Frontend Integration Guide

Audience: Mobile/web frontend engineers and coding agents implementing Sign In, Sign Up, OTP, session, and account deletion.

Service: NestJS HTTP API (`user-service`)

Base URL (local): `http://localhost:3000`

Content-Type: `application/json`

Document version: 1.0

Date: 19 September 2026

This document is the contract. Integrate **only** these routes. Map UI copy to the `error` field (stable codes), not to HTTP status alone.

1. How to use this document (for AI agents)

1. Treat this file as the source of truth for auth/user HTTP APIs.
 2. Never invent extra endpoints, query params, or body fields. Unknown JSON keys are rejected (`forbidNonWhitelisted`).
 3. Drive UI from `error` codes in the JSON body (e.g. `OTP_INVALID` , `PHONE_NOT_REGISTERED`).
 4. Persist `sessionId` between OTP screens. Persist `accessToken` after verify. Send the token as `Authorization: Bearer <accessToken>` .
 5. The user account is **not** created at signup start. It is created only after a successful `POST /auth/otp/verify` for a signup session.
-

2. Product flows (what the app must do)

2.1 Sign up

1. Optional: debounce `GET /users/username/available?username=` while the user types.
2. User submits username (optional), phone (E.164), and accepts terms.
3. `POST /auth/signup` → OTP is sent. Save `sessionId` , `expiresAt` , `resendAvailableAt` , `attemptsRemaining` .
4. OTP screen: 6 digit boxes. Enable Verify only when all 6 digits are entered.
5. `POST /auth/otp/verify` with `sessionId` + `code` .
6. Success → store `accessToken` + `user` → home ("Successful Sign up").
7. Errors map to Figma states (incorrect, expired, locked, resend). See §7.

If username is omitted or blank, the server generates one (`user-1837` style). If provided, it must be unique and valid.

2.2 Sign in

1. User enters phone → `POST /auth/login`.
2. Unknown phone → `PHONE_NOT_REGISTERED` → “This account is not previously registered” / go to Create Account.
3. Known phone → same OTP screen as signup (same verify/resend APIs).
4. Success → store token → home (“Logged in successfully”).

2.3 Phone already registered (from signup)

`POST /auth/signup` with an existing phone → `PHONE_ALREADY_REGISTERED` → “This account is already registered” / Log in instead.

2.4 Log out

Confirm in UI, then `POST /auth/logout` with Bearer token. Discard the token locally. That token will return `401` afterward.

2.5 Delete account (App Store)

`DELETE /auth/account` with Bearer token. User and OTP sessions are removed. Subsequent login with that phone is `PHONE_NOT_REGISTERED`.

3. Conventions

3.1 Phone

- Format: **E.164**, `+` then country code then subscriber number.
- Pattern: `^\+[1-9]\d{7,14}$`
- Examples: `+96651234567`, `+923309890496`
- Spaces are stripped server-side. Do not send dashes or `00` prefixes.
- One phone = one account.

3.2 Username

Rule	Value
Length	3–20
Allowed	letters, numbers, <code>_</code> , <code>-</code>
Pattern	<code>^[A-Za-z0-9_-]+\$</code>
Unique	Case-insensitive check

Rule	Value
Optional on signup	If missing, server generates <code>user-{4 digits}</code>

3.3 OTP

Rule	Value
Length	6 digits (<code>/^\d{6}\$/</code>)
Lifetime	2 minutes (<code>expiresAt</code>)
Verify attempts	3 then logout
Lockout	15 minutes (<code>lockedUntil</code>), per phone (cannot bypass by starting a new login/signup)
Resend cooldown	30 seconds (<code>resendAvailableAt</code>)
Max resends	3 per OTP session

After lockout expires, resend is allowed again and attempts reset.

3.4 Auth header

```
Authorization: Bearer <accessToken>
```

JWT lifetime: **7 days**. Logout increments `tokenVersion`; old tokens become `401`.

3.5 HTTP status (Nest defaults)

Method	Typical success
GET	200
POST	201
DELETE	200

Do not rely on 201 vs 200 for business logic. Rely on JSON body + `error` codes.

3.6 Validation failures (class-validator)

If the body/query fails DTO validation (wrong types, extra fields, bad UUID, phone not E.164):

```
{
  "message": ["phone must be in E.164 format"],
  "error": "Bad Request",
  "statusCode": 400
}
```

`message` may be a string or an array of strings. This is **not** an `error` code like `OTP_INVALID`.

3.7 Business errors (use these in the UI)

```
{
  "error": "OTP_INVALID",
  "message": "Incorrect code. Please check the digits and try again.",
  "attemptsRemaining": 2
}
```

Always read `error` (string code). Extra fields may include `attemptsRemaining`, `lockedUntil`, `resendAvailableAt` (ISO-8601).

3.8 Unauthorized

Missing/invalid/logged-out token:

```
{
  "message": "Unauthorized",
  "statusCode": 401
}
```

4. Route index

Method	Path	Auth	Purpose
GET	<code>/health</code>	No	Liveness
GET	<code>/users/username/available</code>	No	Username uniqueness
POST	<code>/auth/signup</code>	No	Start signup OTP
POST	<code>/auth/login</code>	No	Start login OTP
POST	<code>/auth/otp/verify</code>	No	Verify OTP; create/login user; return JWT
POST	<code>/auth/otp/resend</code>	No	New OTP for the same session
GET	<code>/auth/me</code>	Bearer	Current user
POST	<code>/auth/logout</code>	Bearer	Invalidate JWT
DELETE	<code>/auth/account</code>	Bearer	Delete account

There is **no** TCP/microservice API. HTTP only.

5. Endpoints in detail

5.1 GET /health

Auth: none

Success 200

```
{
  "status": "ok",
  "service": "user-service",
  "timestamp": "2026-09-19T17:00:00.000Z"
}
```

5.2 GET /users/username/available

Auth: none

Query

Param	Required	Rules
username	yes	3-20, [A-Za-z0-9_-]+

Example: /users/username/available?username=Murabaa-1837

Success 200

```
{ "available": true }
```

```
{ "available": false }
```

Errors

HTTP	error	When	UI
400	USERNAME_INVALID	Too short/long or illegal characters	Inline username validation (red rules)

Example:

```
{
  "error": "USERNAME_INVALID",
  "message": "Username must be 3-20 characters: letters, numbers, underscores, or hyphens."
}
```

Call this while typing (debounced). Do not block the keyboard on network; show loading on the field.

5.3 POST /auth/signup

Auth: none

Body

```
{
  "phone": "+96651234567",
  "username": "Murabaa-1837",
  "acceptedTerms": true
}
```

Field	Type	Required	Notes
phone	string	yes	E.164
username	string	no	If omitted, server generates
acceptedTerms	boolean	yes	Must be true

Success 201 — OTP session started (user **not** created yet)

```
{
  "sessionId": "a44feb09-4afd-48d6-9300-b465acc18b1c",
  "expiresAt": "2026-09-19T17:14:15.056Z",
  "resendAvailableAt": "2026-09-19T17:12:45.056Z",
  "attemptsRemaining": 3
}
```

Field	Meaning
sessionId	UUID. Required for verify and resend
expiresAt	ISO time when this OTP dies (2 min)
resendAvailableAt	ISO time when Resend may be enabled (30s)
attemptsRemaining	Verify tries left (starts at 3)

Errors

HTTP	error	UI
400	TERMS_REQUIRED	Terms checkbox
400	USERNAME_INVALID	Username rules

HTTP	error	UI
409	USERNAME_TAKEN	Username already used
409	PHONE_ALREADY_REGISTERED	"This account is already registered" → Log in
429	OTP_LOCKED	Phone is in 15-min lockout (lockedUntil)
400	validation	Bad phone / extra fields

5.4 POST /auth/login

Auth: none

Body

```
{
  "phone": "+96651234567"
}
```

Success 201 — same shape as signup OTP session (sessionId , expiresAt , resendAvailableAt , attemptsRemaining).

Errors

HTTP	error	UI
404	PHONE_NOT_REGISTERED	"This account is not previously registered" → Sign up
429	OTP_LOCKED	Temporarily locked (lockedUntil)
400	validation	Bad phone

5.5 POST /auth/otp/verify

Auth: none

Body

```
{
  "sessionId": "a44feb09-4afd-48d6-9300-b465acc18b1c",
  "code": "955641"
}
```

Field	Rules
sessionId	UUID from signup/login/resend

Field	Rules
code	exactly 6 digits, string

Success 201 — account created (signup) or logged in (login)

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "8f14e45f-ea8d-4c3b-9c1a-2b7e0d1a9c11",
    "username": "Murabaa-1837",
    "phone": "+96651234567"
  }
}
```

Store both. Attach `accessToken` to later requests.

Errors

HTTP	error	Extra	UI
400	OTP_INVALID	attemptsRemaining	Red digits, "Incorrect code... Send a new code?"
400	OTP_EXPIRED		"The code has expired. Send a new code"
429	OTP_LOCKED	lockedUntil	"Temporarily locked... try again after 15:00 minutes"
404	SESSION_NOT_FOUND		Session missing or already used
400	validation		Code not 6 digits / bad UUID

On the last failed attempt the API returns `OTP_LOCKED` (not `OTP_INVALID` with 0 remaining).

Yellow "You have only 3 attempts" can be shown from `attemptsRemaining === 3` on first OTP screen, or whenever you want a warning; the server always returns remaining on invalid verify.

5.6 POST /auth/otp/resend

Auth: none

Body

```
{
  "sessionId": "a44feb09-4afd-48d6-9300-b465acc18b1c"
}
```

Success 201 — same session payload as signup (`sessionId` unchanged, new `expiresAt` / `resendAvailableAt`).

Disable the Resend control until `resendAvailableAt` . After expiry of the code, still use this endpoint (not a new signup) unless the session is gone.

Errors

HTTP	error	Extra	UI
429	OTP_RESEND_COOLDOWN	<code>resendAvailableAt</code>	Keep Resend disabled
429	OTP_RESEND_LIMIT		No more resends this session
429	OTP_LOCKED	<code>lockedUntil</code>	Lockout; Resend disabled until then
404	SESSION_NOT_FOUND		Start signup/login again

5.7 GET /auth/me

Auth: Bearer

Success 200

```
{
  "id": "8f14e45f-ea8d-4c3b-9c1a-2b7e0d1a9c11",
  "username": "Murabaa-1837",
  "phone": "+96651234567"
}
```

Errors: 401 if token missing, invalid, expired, or logged out.

5.8 POST /auth/logout

Auth: Bearer

Success 201

```
{ "ok": true }
```

Then delete the token from device storage. Do not reuse it.

Errors: 401

5.9 DELETE /auth/account

Auth: Bearer

Success 200

```
{ "ok": true }
```

Permanent. Confirm in UI first.

Errors: 401

6. Suggested client types

```
export type OtpSessionResponse = {
  sessionId: string;
  expiresAt: string; // ISO-8601
  resendAvailableAt: string;
  attemptsRemaining: number;
};

export type PublicUser = {
  id: string;
  username: string;
  phone: string;
};

export type AuthSuccessResponse = {
  accessToken: string;
  user: PublicUser;
};

export type ApiErrorBody = {
  error?: string;
  message: string | string[];
  statusCode?: number;
  attemptsRemaining?: number;
  lockedUntil?: string;
  resendAvailableAt?: string;
};
```

7. Error code catalog (bind UI to these)

error	HTTP	Extra fields	Suggested UI
USERNAME_INVALID	400		Username rules failed
USERNAME_TAKEN	409		Username not unique
PHONE_NOT_REGISTERED	404		Login: account does not exist
PHONE_ALREADY_REGISTERED	409		Signup: go to login
TERMS_REQUIRED	400		Must accept terms
SESSION_NOT_FOUND	404		Restart OTP flow
OTP_INVALID	400	attemptsRemaining	Incorrect code
OTP_EXPIRED	400		Code expired; resend
OTP_LOCKED	429	lockedUntil	15 minute lock
OTP_RESEND_COOLDOWN	429	resendAvailableAt	Wait to resend
OTP_RESEND_LIMIT	429		Max 3 resends

ISO timestamps are UTC (Z). Convert to local countdown in the UI.

8. Integration sequence diagrams

Sign up

```
Client                                API
| GET username/available              |
|----->|
| { available: true }                 |
|<-----|
| POST /auth/signup                   |
| { phone, username, terms }          |
|----->|
| { sessionId, expiresAt... }         |
|<-----|
| POST /auth/otp/verify               |
| { sessionId, code }                 |
|----->|
| { accessToken, user }               |
|<-----|
| GET /auth/me (Bearer)               |
|----->|
```

Sign in

```
POST /auth/login { phone }  
→ 404 PHONE_NOT_REGISTERED OR OTP session  
POST /auth/otp/verify { sessionId, code }  
→ { accessToken, user }
```

9. Frontend implementation notes

1. **Do not** expect the OTP in the HTTP response. It is sent by SMS (Twilio) or logged on the server in development.
2. Countdown timers should use `expiresAt` / `resendAvailableAt` / `lockedUntil` from the server, not a hardcoded client clock only.
3. After 3 wrong codes, disable Verify and Resend until `lockedUntil`.
4. `forbidNonWhitelisted`: sending `countryCode` or `otp` extra keys causes 400 validation errors. Only send documented fields.
5. Username availability is **advisory**; signup is the source of truth (`USERNAME_TAKEN` race is possible).
6. Health is optional for the app; useful for debugging.
7. Privacy Policy / Terms URLs are **not** served by this API. Render them in-app. You only send `acceptedTerms: true`.
8. Location permission (iOS) is client-side; this API does not accept lat/lng.
9. Optional email recovery is **not** implemented.

10. Example requests

Signup

```
POST /auth/signup HTTP/1.1  
Host: localhost:3000  
Content-Type: application/json  
  
{ "phone": "+923309890496", "username": "MusharafMobeen", "acceptedTerms": true }
```

Verify

```
POST /auth/otp/verify HTTP/1.1  
Host: localhost:3000  
Content-Type: application/json  
  
{ "sessionId": "a44feb09-4afd-48d6-9300-b465acc18b1c", "code": "955641" }
```

Authenticated

```
GET /auth/me HTTP/1.1  
Host: localhost:3000  
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

11. Out of scope (do not call)

- No GraphQL, no gRPC, no TCP message patterns
 - No password, no email login, no social login
 - No refresh-token endpoint (access JWT only)
 - No "get OTP" HTTP endpoint
 - No profile update / email save in this version
-

End of Metr User Service frontend integration contract.